

UST-2026-09-26-19

Alta pública blindada: la lista de espera se apunta por una puerta (Edge waitlist-join tras /api/waitlist en nuestro dominio) con honeypot, ritmo por IP, Origin y Turnstile; el INSERT anónimo por REST se retira; solo cuenta quien confirma su correo (doble opt-in); el digest avisa del crecimiento

PROPUESTA VALIDADA EN PARTE · alcance dado por Juanjo en el chat (26-sep: «2+3+4+5+7» y, al leer la propuesta, «hay que meter también 6» → las SEIS capas: 2+3+4+5+6+7) · D1-D9 DECIDIDOS por Juanjo (26-sep, ≈23:50: «D1 a D9, se implementan según las recomendaciones, no alternativas») · D10 y D14 (técnicas) pendientes de una palabra · D11-D13 → firma de Pilar y Lucía (□ WL-2). Firma electrónica simple pendiente. Nada construido. Lo que se afirma está leído del código (repos/web2/src/js/30-forms.js, tools/patches/waitlist.py, integrate.py ns-waitlist-js, Edge community-lead) o medido en la base el 26-sep. La capa 6 (doble opt-in) toca qué guardamos y cuánto tiempo → carril producto: la firman Pilar y Lucía (□ WL-2, D11-D13).

1 · El pedido, literal

Juanjo, 26-sep 2026, ≈23:00, tras cerrar el incidente de la waitlist (UST-18):

«como se puede prevenir que un bot meta esos correos falsos»

«2+3+4+5+7» (sobre las siete capas propuestas: 2 cerrar REST y pasar por Edge · 3 honeypot, ritmo por IP, Origin, tiempo mínimo · 4 proxy en nuestro dominio con WAF/rate limiting de Cloudflare · 5 Turnstile · 7 vigilancia diaria)

«hay que meter también 6. Doble opt-in: solo cuenta quien confirma por correo; lo no confirmado en 7 días se borra solo. Es la única capa que distingue un correo REAL de uno con formato válido, y de paso es la prueba de consentimiento que Audidat querrá. Toca datos de usuarias (carril producto, firma de las dos) y la cuota de IONOS (~50 correos seguidos = «450 limit») → cola a 5/h como la wave-2.» (Juanjo, tras leer la v1 de este documento)

Lectura: que apuntarse a la lista de espera sea un CAMINO ÚNICO y vigilado (nuestro dominio → Cloudflare → Pages Function → Edge → base), que el camino viejo (POST directo a PostgREST con la anon key) deje de existir, y que una fila solo CUENTE cuando una persona real ha pulsado el enlace de su correo. Un bot no se «disuade»: se le quita la puerta; y lo que aun así entre sin dueño se borra solo a los 7 días.

Principio (stack lean, Juanjo 2-ago)

Sin proveedores nuevos: todo con Cloudflare (Pages Functions, WAF, Turnstile) y Supabase (Edge, SQL) que ya tenemos. Cero servidores propios. Coste: 0 €.

2 · Lo que hay hoy (leído del código y medido en la base)

Qué	Medido
Cómo se apunta alguien	web2 (`src/js/30-forms.js`): `fetch(SB + "/rest/v1/waitlist")` con la anon key y `{email, source:"web2"}`; sin JS, el ` <form>` (`tools/patches/waitlist.py`) hace POST directo a `https://nebkqncvapelrarruyqb.supabase.co/rest/v1/waitlist`. El marketing viejo en `/v1/` (`integrate.py` bloque `ns-waitlist-js`) hace lo mismo con `marketing_site`; el pie, `newsletter`. La cifra de la Home sale de la RPC `waitlist_count` (anon).</form>
La anon key	Pública por diseño (RLS manda) y visible en el HTML de la web: cualquiera puede llamar al endpoint sin pasar por el formulario. Eso hizo el bot: 630.076 POST en dos ventanas, sin abrir la web.
Guardas de hoy (UST-18)	Tope de lote (>20 filas/sentencia) · origen obligatorio para anon (web2 / marketing_site / newsletter) · ritmo GLOBAL 200 altas/10 min · CHECK de formato · `created_at` lo pone el servidor. Probadas en producción (curl → 400/23514).
Lo que las guardas NO ven	La IP (un trigger no la tiene de forma fiable), si hay un humano detrás, si la petición viene de nuestra web. Y el ritmo es global: 200/10 min frenaría también un pico legítimo.
Lo que Cloudflare ve hoy	Nada del alta: el POST va del navegador (o del bot) directo a `supabase.co`. WAF, rate limiting y bots de Cloudflare protegen nutrisynccollective.com, no el endpoint.

Qué	Medido
El patrón ya existe en casa	Edge `community-lead` (5/6-sep): honeypot `website`, ritmo por email (3/24 h) y por hash de IP (10/h) vía RPC `community_lead_allowed`, Origin en CORS, 303 → página de gracias sin JS, nunca guarda la IP (sha256 truncado). `waitlist-join` es su gemela.
Vigilancia	Ninguna: 630.083 filas estuvieron 16 días sin que nadie las viera. El digest de las 07:15 (`founder_pending_summary`) no mira la waitlist.
Real vs válido	Nada distingue hoy un correo real de uno con formato válido pero ajeno: la fila entra y cuenta en el «160+» de la Home en el acto. El consentimiento es la casilla del formulario (web2 `input[name=consent]`), sin prueba de que el buzón exista ni de que su dueña lo pidiera. `waitlist` tiene `converted_at` y `locale` (idioma del alta), pero ninguna columna de confirmación.
Correo saliente	SMTP de IONOS (`feedback@nutriscynccollective.com`, reply-to `contact@`): cuota que se agota a ~50 envíos seguidos («450 Mail send limit exceeded», r15); la wave-2 se drenó con un cron horario a 5 correos por corrida (`nutriscync-wave2-drain`, sello por correo). Ese patrón ya existe y se reutiliza.

Analogía: hoy la base de datos tiene un puerto publicado a Internet con una ACL por fila (RLS). La arquitectura buena es la de cualquier API: un API gateway en nuestro dominio (Pages Function detrás del WAF de Cloudflare) que autentica la petición (Turnstile = «hay un humano»), la limita (rate limit por IP), la valida (honeypot, Origin, formato) y la reenvía al backend por un canal que solo él conoce (la Edge con un secreto compartido); y el puerto del backend deja de escuchar a Internet (se retira la policy de INSERT anónimo).

3 · Cambios, fila a fila (propuesta)

Qué	Qué se construirá	Cómo se verificará
C1 · Edge `waitlist-join`	Gemela de `community-lead`: acepta JSON (fetch) o form-urlencoded (sin JS); campos `email`, `source` (solo de la lista blanca: web2 / marketing_site / newsletter), `locale`, `website` (honeypot), `cf-turnstile-response`, `t0` (instante de carga del formulario). Rechaza si: honeypot relleno (responde OK y no guarda), Origin fuera de nuestros dominios, formulario enviado < 2 s tras cargarse, Turnstile inválido (cuando hay token), o `waitlist_allowed(email, ip_hash)` dice no. Inserta con service_role (Vault). Exige la cabecera `x-ns-gateway` (secreto compartido con la Pages Function): sin ella, 403 — así nadie llama a la Edge saltándose Cloudflare. 409 si el correo ya está; 303 → gracias/error sin JS; JSON `{ok}` con JS.	Arnés de funciones puras (transpilar con el `typescript` de appdev, como community-lead) para: lista blanca de source, honeypot, tiempo mínimo, forma del hash de IP. Estreno REAL con una llamada por cada camino (r21): fetch OK · sin JS 303 · honeypot 200-sin-fila · sin gateway 403 · Turnstile inválido 400 · duplicado 409.
C2 · SQL: ritmo por IP y cierre del REST	`waitlist_allowed(p_email, p_ip_hash)` security definer (≤ 3 altas/24 h por email · ≤ 5/h por hash de IP; sin JS/Turnstile, ≤ 3/h) sobre `waitlist_ritmo` (ip_hash, email_hash, created_at; purga > 48 h en la propia función). `drop policy "anyone can join waitlist" + `revoke insert on waitlist from anon` (authenticated conserva el grant para la policy `admins * waitlist` del hub). Las guardas de UST-18 se quedan (defensa en profundidad; la Edge entra como service_role y no las pisa).	Fichero con `begin/commit`, autoprueba (`set local role anon` → INSERT = 42501 permission denied) y verificación final. Arnés PG16 antes de la bandeja. Se ejecuta EL ÚLTIMO (D10): cuando la web ya envía por la puerta nueva.
C3 · Pages Function `/api/waitlist` (webdeploy)	`repos/webdeploy/functions/api/waitlist.js`: recibe el POST (JSON o form), añade `cf-connecting-ip` como `x-ns-ip` y `x-ns-gateway` (secreto en variable de entorno del proyecto Pages), reenvía a la Edge y devuelve su respuesta (JSON o 303). Misma URL para web2 (raíz), `/v1/` y m. Ventaja: mismo origen (sin CORS) y la anon key sale del JS de la web.	web-tests: el módulo se ejecuta en vm con peticiones de forma (JSON, form, honeypot) contra una Edge falsa. smoke-prod: `POST /api/waitlist` con honeypot → 200 `{ok:true}` y `waitlist_count` NO cambia.
C4 · Cloudflare: WAF y rate limiting (Juanjo, panel)	Security → WAF → Rate limiting rules: `http.request.uri.path eq "/api/waitlist"` → 10 peticiones / 10 s por IP → bloquear 10 s (la regla gratuita del plan). Bot Fight Mode: NO (D6) — desafiaría también a smoke-prod, curl y Lighthouse, y en el plan Free no admite excepciones.	Prueba con `ab`/bucle de curl desde el Mac: la 11ª petición en 10 s devuelve 429 de Cloudflare; la Edge no la ve.

Qué	Qué se construirá	Cómo se verificará
C5 · Turnstile (Cloudflare) en los formularios	Widget «Managed» (invisible salvo sospecha) en los 3 formularios de alta (web2 Home + Community, `v1/`, pie newsletter). Site key en el HTML (pública); secret en `supabase secrets` para la Edge, que verifica en `challenges.cloudflare.com/turnstile/v0/siteverify` con la IP. El script `https://challenges.cloudflare.com/turnstile/v0/api.js` es una DESVIACIÓN DECLARADA de po74 («cero CDNs de terceros»); mismo proveedor que ya sirve la web y ya ve todas las peticiones; lista blanca explícita, con el motivo, en pre-deploy (bloque 7), verify-web-pack y smoke. Pilar corrobora y, si procede, una línea en la política de privacidad.	Capturas ES/EN × 390/1440 (textfit no debe cambiar: el widget va debajo del botón). En vivo: alta con token válido → 201; sin token con JS → 400 «verificación pendiente»; sin JS → camino C1 con límite estricto.
C6 · web2 y `v1/`: el formulario habla con `api/waitlist`	`30-forms.js`: `fetch("/api/waitlist")` (mismo origen), cuerpo `{email, source, locale, website, t0, cf-turnstile-response}`; `tools/patches/waitlist.py`: `action="/api/waitlist"` (sin JS). `integrate.py` bloque `ns-waitlist-js` (marketing viejo en `v1/`): misma URL. Mensajes ya existentes (ok/dup/err) + uno nuevo «verificación pendiente».	verify-web2 + textfit en 5 idiomas × 2 anchos · candidata `v2.` verificada por Juanjo (circuito de 5 pasos, «sin 2 y 3 no hay 4 o 5») · web-tests (snippet `ns-waitlist-js` real en vm).
C7 · Vigilancia en el digest de las 07:15	`founder_pending_summary` añade `waitlist_altas 24h` y `waitlist_total`; `founder-reminders` escribe una línea siempre («  50. Además, una sonda de solo lectura re-ejecutable (`-sonda-waitlist-crecimiento.sql`)).	Estreno con una llamada real a la Edge (mirar el cuerpo, pero EN/ES/CA/GL/EU en 24h se muestra 60 la marca  altas en 24 h → aparece el  .
C8 · Orden de entrega A (el formulario nunca se queda sin puerta)	(1) SQL de `waitlist ritmo` + `waitlist allowed` → (2) `supabase secrets` (Turnstile secret + gateway) y Deploy Edge con verificación en vivo → (3) variable `NS_GATEWAY` en Pages (nutrisync-collective) → (4) web2 candidata → verificada → integrate → Deploy Web (po) → (5) smoke → (6) SOLO ENTONCES el SQL que retira la policy y el grant → (7) smoke otra vez: POST anon a `rest/v1/waitlist` = 401/42501.	Cada paso con su comprobación antes del siguiente (regla 7-sep: validar antes de sobrescribir). Baseline antes del paso 1.
C9 · Doble opt-in: solo cuenta quien confirma (capa 6 — entrega B, tras la firma de  WL-2)	SQL: `waitlist.confirmed_at timestampz`, `confirm_token uuid default gen_random_uuid()` (un solo uso), `confirm_sent_at timestampz`. La Edge `waitlist-join` inserta con `confirmed_at = null` y NO envía nada en línea: la fila es la cola. Cron horario `nutrisync-waitlist-confirm` (min 50) → Edge `waitlist-confirm-drain`: ≤5 correos por corrida (patrón `nutrisync-wave2-drain`: sello `confirm_sent_at` POR CORREO justo tras enviar; el motivo SMTP viaja en la respuesta), remitente `feedback@`, reply-to `contact@`, texto en el idioma de la fila (`locale`: EN/ES/CA/GL/EU) con el enlace `https://nutrisynccollective.com/confirm?t=<token>`. Pages Function `/confirm` → Edge `waitlist-confirm`: token válido y no caducado (7 días desde el envío) → `confirmed_at = now()`, token anulado → página «¡Confirmado!» (5 idiomas); caducado o desconocido → página con motivo y botón «volver a enviarme el enlace» (misma cola, ritmo por email). `waitlist_count()` cuenta SOLO confirmadas. `admin_waitlist` devuelve `confirmed_at`;  pinta la insignia «pendiente de confirmar» y el total de confirmadas.	Arnés PG16: la purga y `waitlist_count` con filas confirmadas/pendientes/caducadas. Estreno REAL (r21/r20): un alta propia → el cron (o una llamada manual `x-ns-cron` con `lote:1`) envía UN correo → se mira en la bandeja → clic → `confirmed_at` sellado y la Home sube en 1. Un token repetido → «ya confirmado»; uno inventado → «enlace no válido». Nunca ráfagas de prueba (quema cuota).
C10 · Purga de lo no confirmado y las 168 de siempre	Purga en la propia función de drenaje (una pasada por corrida, sin cron aparte): `delete from waitlist where confirmed_at is null and confirm_sent_at < now() - 7 días` — se cuenta desde el ENVÍO, no desde el alta: una fila a la que la cuota no dejó escribir no se borra sin haber tenido su oportunidad. Las 168 filas anteriores al doble opt-in se marcan `confirmed_at = created_at` en el mismo SQL (D12): se apuntaron antes de la regla y ya han recibido correos nuestros; re-pedirles confirmación costaría 168 envíos (la cuota de IONOS son dos días) y perdería a quien no abra el correo. Cada purga deja una línea en el digest («borradas N sin confirmar»).	Arnés: sembrar filas en los cuatro estados (confirmada · pendiente reciente · pendiente > 7 días con envío · pendiente sin envío) → solo se borra la tercera. En producción: la línea del digest del día siguiente.

4 · Análisis de impacto

Dónde	Qué toca	Qué NO toca
repos/web2	`src/js/30-forms.js`, `tools/patches/waitlist.py` (+ Turnstile en la barra), catálogo i18n (1 clave × 5 idiomas), `build.py` sin cambios de forma → circuito de 5 pasos completo.	Layout, imágenes, Community (sus formularios ya van por Edge), SEO, PWA shell.

Dónde	Qué toca	Qué NO toca
repos/webdeploy	Nuevo `functions/api/waitlist.js`; `integrate.py` bloque `ns-waitlist-js` (URL) y lista blanca de Turnstile en pre-deploy/verify-web-pack; `_routes.json` si hace falta para `/api/*`.	Hub (□ Waitlist y □ Piloto siguen con `sb.from("waitlist")` como admins; policy `admins * waitlist` intacta), legal, adjuntos, middleware pages.dev.
Backend SQL	Entrega A: `waitlist_ritmo` (nueva), `waitlist_allowed` (nueva), `founder_pending_summary` (v+1), drop policy + revoke insert anon. Entrega B: columnas `confirmed_at` / `confirm_token` / `confirm_sent_at` en `waitlist`, `waitlist_count()` solo confirmadas, `admin_waitlist` con `confirmed_at`, marcado de las 168 (D12), cron `nutrisync-waitlist-confirm`. Las guardas de UST-18 se quedan.	Las filas de `waitlist` (ninguna se borra en la entrega; la purga solo toca pendientes con envío > 7 días). `admin_waitlist` paginada = item aparte S19.
Edge Functions	Nuevas `waitlist-join` (A), `waitlist-confirm-drain` y `waitlist-confirm` (B); `founder-reminders` (dos líneas nuevas: altas 24 h pendientes/borraduras). Secrets: `TURNSTILE_SECRET`, `NS_GATEWAY_SECRET` (el SMTP ya está).	`community-lead` (se copia, no se toca), `notify-dispatch`, los crons existentes.
Cloudflare (panel, Juanjo)	Turnstile: crear widget (site key + secret). WAF: 1 regla de rate limiting. Pages: variable de entorno `NS_GATEWAY`. Access: nada.	DNS, Zero Trust, Bot Fight Mode (NO se activa, D6).
Correo (IONOS)	Entrega B: un correo de confirmación por alta, drenado a ≤5 por hora (máx. 120/día) desde `feedback@`; texto en 5 idiomas (copy de Lucía, D13). Los rebotes llegan al buzón `feedback@` y no se procesan (lean): la purga de 7 días hace ese trabajo.	Los recordatorios del piloto, el digest de las 07:15, los acuses de Community (mismas credenciales, misma cuota: por eso el ritmo).
Quién lo recibe	Quien se apunte desde la web (EN/ES/CA/GL/EU) y desde `/v1/`: mismo gesto, un widget más (normalmente invisible) y, con B, un correo con un enlace que confirmar. Las founders: dos líneas nuevas en el digest y la insignia «pendiente» en □.	Testers y usuarias de la app (la app no escribe en waitlist). Las 168 personas ya apuntadas (D12: se dan por confirmadas, no reciben nada).
Riesgos	Turnstile mal configurado = nadie se apunta → por eso el orden C8 y el camino sin token con límite estricto (D4) como red. Un fallo en la Edge = 303 a la página de error con motivo (nsFail, nada en silencio). La regla WAF demasiado agresiva = 429 a humanos rápidos (10/10 s por IP es holgado para una persona). Con B: un correo que llega a spam = fila sin confirmar que se borra a los 7 días → el texto lleva «busca NutriSync en spam» y el botón de reenvío en la página de caducado; DKIM/DMARC ya publicados (31-ago). Un pico legítimo (prensa) espera en la cola: a 5/h, 500 altas tardan 4 días en confirmarse — se avisa en el digest y se sube el lote a mano si hace falta.	—

Desviación declarada (única): el script de Turnstile se carga de `challenges.cloudflare.com`. po74 exige cero CDNs de terceros; Cloudflare no es un tercero nuevo (sirve la web entera desde Pages y ya ve cada petición), pero es un script externo y se registra como excepción con motivo en las tres puertas. Pilar corrobora (compliance) y decide si la política de privacidad lo menciona. Carriles: D1-D10 y D14 son técnicos (Juanjo); D11-D13 tocan qué guardamos, cuánto tiempo y qué se le escribe a una persona → carril producto: □ WL-2, firma de Pilar y Lucía ANTES de construir la entrega B.

5 · Decisiones (fila a fila — D1-D9 decididas por Juanjo el 26-sep; D10 y D14 pendientes; D11-D13 las firman Pilar y Lucía)

#	Propuesta	Alternativa	Decisión
D1	Cerrar el REST para anon: `drop policy "anyone can join waitlist" + `revoke insert ... from anon`. El POST directo a `/rest/v1/waitlist` pasa a 401/42501 para todo el mundo.	Dejar el REST abierto y añadir solo Turnstile en el HTML (no cierra nada: el bot no usa el HTML).	DECIDIDO (Juanjo, 26-sep): la propuesta — se cierra el REST para anon.
D2	Puerta ÚNICA `/api/waitlist` en nuestro dominio (Pages Function) → Edge con `x-ns-gateway` (secreto compartido); la Edge rechaza (403) lo que no venga por la puerta. Así Cloudflare ve y limita TODO el tráfico de alta.	Edge pública como `community-lead` (llamable directo desde el navegador: Cloudflare no la ve y el WAF no aplica).	DECIDIDO: la propuesta — puerta única `/api/waitlist` + secreto de gateway en la Edge.

#	Propuesta	Alternativa	Decisión
D3	Turnstile modo «Managed» (invisible salvo sospecha), verificado en la Edge con la IP; site key pública en el HTML; desviación po74 declarada con lista blanca y motivo; Pilar corrobora.	Sin Turnstile (solo honeypot + ritmo). Más simple, pero un bot con navegador real y una IP distinta por petición sigue entrando al ritmo permitido.	DECIDIDO: la propuesta — Turnstile «Managed»; desviación po74 declarada (Pilar corrobora).
D4	Sin JS (o Turnstile caído): se ACEPTA el alta con honeypot + Origin + ritmo estricto (3/h por IP) — accesibilidad y red de seguridad si Turnstile falla.	Rechazar todo alta sin token de Turnstile.	DECIDIDO: la propuesta — sin JS se acepta con ritmo estricto.
D5	Ritmo por IP con hash sha256 truncado (nunca la IP en claro, como community-lead): 5/h por IP · 3/24 h por email; tabla `waitlist_ritmo` purgada a las 48 h dentro de la propia función.	Solo la regla de rate limiting de Cloudflare (por IP real, sin memoria entre peticiones lentas: 1 alta cada 11 s la pasa).	DECIDIDO: la propuesta — ritmo por hash de IP (5/h) y por email (3/24 h).
D6	Regla WAF de rate limiting: `/api/waitlist`, 10 peticiones/10 s por IP → bloqueo 10 s. Bot Fight Mode NO (desafiaría a smoke-prod, curl, Lighthouse; en Free no admite excepciones).	Bot Fight Mode ON en toda la zona.	DECIDIDO: la propuesta — regla WAF 10/10 s; Bot Fight Mode NO.
D7	El tope global de UST-18 (200/10 min) se queda como red de seguridad de la base (defensa en profundidad), igual que el tope de lote y la guarda de origen.	Retirarlo al cerrar el REST.	DECIDIDO: la propuesta — las guardas de UST-18 se quedan.
D8	Vigilancia: línea SIEMPRE en el digest de las 07:15 («+N en 24 h · total T») y Δ si $N > 50$; sonda re-ejecutable en backend-sql.	Cron propio + correo aparte (más piezas, mismo aviso).	DECIDIDO: la propuesta — línea en el digest + Δ si > 50 ; sonda re-ejecutable.
D9	Carril técnico: Juanjo decide y ejecuta; Pilar corrobora la desviación po74 (Turnstile) y el texto de privacidad si procede; Lucía, el widget bajo el botón (no cambia el diseño).	Carril producto (firma previa de las dos).	DECIDIDO: la propuesta — carril técnico; Pilar corrobora D3.
D10	Orden C8: la web habla con la puerta nueva ANTES de cerrar la vieja; `drop policy` es el ÚLTIMO paso, con smoke antes y después. Entrega A (capas 2-5+7) en S19 ($\approx$ 4-5 h de trabajo + los 5 pasos del circuito web2); entrega B (capa 6) cuando WL-2 esté firmada ($\approx$ 3-4 h más).	Cerrar primero el REST (deja el formulario roto entre pasos) · o todo en una sola entrega esperando a WL-2.	
D11	☐ WL-2 · Doble opt-in: solo cuenta quien confirma por correo (enlace de un uso, 7 días); lo no confirmado se borra solo a los 7 días del envío; el número público de la Home cuenta solo confirmadas. Es la prueba de consentimiento (fecha + token) que pedirá Audidat.	Sí, pero sin borrar (lo no confirmado se marca y no cuenta) · No: basta con las capas 2-5.	Carril PRODUCTO: firman Pilar y Lucía (☐ WL-2).
D12	Las 168 filas anteriores a la regla se dan por confirmadas (`confirmed_at` = `created_at`): se apuntaron antes del doble opt-in y ya han recibido correos nuestros; no se les pide nada.	Pedir confirmación a las 168 (168 envíos = dos días de cuota y pérdida segura de quien no abra el correo).	Carril PRODUCTO: firman Pilar y Lucía.
D13	El correo de confirmación: remitente `feedback@` (reply-to `contact@`), asunto y cuerpo cortos en el idioma del alta (EN/ES/CA/GL/EU), un solo enlace, «si no fuiste tú, ignora este correo», «busca NutriSync en spam» — copy de Lucía sobre la plantilla de ingeniería; cola a 5/h.	Plantilla de ingeniería tal cual · solo EN/ES.	Carril PRODUCTO (texto a una persona): firman Pilar y Lucía; Lucía escribe.
D14	La confirmación vive en `waitlist` (tres columnas) y no en una tabla aparte: una fila, un estado; `converted_at` y `locale` ya viven ahí.	Tabla `waitlist_confirmaciones` con histórico de envíos.	

6 · Baseline, verificación y marcha atrás

Momento	Qué se comprueba
PASO 0 · Baseline (antes de tocar)	`tools/baseline.sh` (sha de web2/webdeploy, sellos ns-build, candados) + lista de policies de `waitlist` (`pg_policies`) + `waitlist_count()` + `supabase functions list`. Sin baseline escrita no hay marcha atrás.
Test antes de desplegar	Arnés de puras de la Edge · arnés PG16 del SQL (incluida la autopruueba «anon INSERT = 42501») · web-tests (Pages Function + snippet ns-waitlist-js) · verify-web2 + textfit ×5 idiomas · pre-deploy bloque 7 con la lista blanca.
Estreno real (r21)	Un POST por camino contra producción: fetch OK (201, aparece en ☐) · sin JS (303 a gracias) · honeypot (200, no aparece) · sin gateway a la Edge (403) · Turnstile inválido (400) · duplicado (409) · 11 peticiones en 10 s (429 de Cloudflare) · POST anon a `/rest/v1/waitlist` (401/42501, tras D1).
DoD entrega A	Las tres verificaciones del párrafo anterior + una persona (Lucía) se apunta desde su móvil en ES y aparece en ☐ · el digest del día siguiente trae la línea de la waitlist.
Estreno real entrega B (r20/r21: se mira el cuerpo y la bandeja)	Un alta propia → `confirmed_at` null y `confirm_sent_at` null → llamada manual al drenaje con `x-ns-cron` y `lote:1` → la respuesta trae `enviados:1` y el correo está en la bandeja (Gmail «Mostrar original»: dkim=pass) → clic → página «¡Confirmado!», `confirmed_at` sellado, la Home sube en 1 → segundo clic → «ya confirmado» → token inventado → «enlace no válido». Purga: en el arnés, nunca en producción con datos reales.
DoD entrega B	Lucía y Pilar se apuntan con un correo propio, reciben el correo en su idioma y confirman; ☐ pasa de «pendiente» a confirmada; el digest del día siguiente trae «pendientes de confirmar: N · borradas: 0».
Marcha atrás	A · SQL: `create policy "anyone can join waitlist" ... with check (true)` + `grant insert on waitlist to anon` (una sentencia cada una; fichero `rollback` preparado). Web: Cloudflare Pages → Rollback al deploy anterior (web2 candidata y webdeploy). Edge: no hace falta retirarla. WAF: desactivar la regla. B · `cron.unschedule` del drenaje + `waitlist_count()` a su versión anterior (las columnas se quedan: no rompen nada); las filas purgadas no vuelven (por eso la purga solo toca pendientes con envío > 7 días y deja rastro en el digest).

7 · What to Test (para Pilar y Lucía, cuando esté desplegado)

- Apuntarse desde la Home (ES y EN) con un correo propio: mensaje de «¡Estás dentro!» y la persona aparece en ☐ Waitlist en el acto (con la entrega B: como «pendiente de confirmar»). Normalmente no se ve ningún «¿eres humano?»; si aparece, es un clic.
- Apuntarse otra vez con el mismo correo: «ya estabas en la lista» (409), sin duplicado en ☐.
- Desde el pie (newsletter) y desde /v1/: lo mismo.
- Entrega B: en menos de una hora llega un correo de NutriSync en tu idioma con un enlace; al pulsarlo, «¡Confirmado!» y en ☐ la fila pasa a confirmada; el número de la Home sube en uno. Pulsarlo otra vez dice «ya confirmado».
- Entrega B: si no confirmas, a los 7 días la fila desaparece sola y el digest lo dice («borradas 1 sin confirmar»). Las 168 de siempre no reciben nada y siguen contando.
- En el digest de las 07:15 del día siguiente: línea «☐ lista de espera: +N en 24 h · total T · pendientes P».
- Para Juanjo: el curl del 26-sep (POST anónimo a /rest/v1/waitlist) devuelve ahora 401/42501 en vez de 400/23514 — la puerta ya no existe, no es que esté vigilada.

8 · La cadena

Paso	Estado
UST	Este documento (propuesta v2, 26-sep). Alcance «2+3+4+5+6+7» validado por Juanjo en chat.
Sign-off	Carril técnico: D1–D9 DECIDIDOS y FIRMADOS por Juanjo el 26-sep («se implementan según las recomendaciones, no alternativas»; firma electrónica simple abajo); D10 y D14 pendientes de una palabra (se firman en el as-built). Carril producto: Pilar y Lucía firman D11–D13 (☐ WL-2) ANTES de construir la entrega B. Pilar corrobora además la desviación po74 (D3) antes del Deploy Web de A.

Paso	Estado
Dev	A (S19): SQL (ritmo + cierre REST) · Edge `waitlist-join` · Pages Function · web2 (formulario + Turnstile) · integrate.py (`ns-waitlist-js`, lista blanca) · founder-reminders · sonda. B (tras WL-2): SQL (columnas, count, admin, 168) · Edge `waitlist-confirm-drain` + `waitlist-confirm` + cron · Pages Function `/confirm` · página de confirmación web2 (5 idiomas) ·  insignia · digest.
Test	Arnés de puras · arnés PG16 (A: policy; B: purga y count en los cuatro estados) · web-tests · verify-web2 + textfit · pre-deploy · estreno real por cada camino (B: UN correo, mirado en la bandeja).
Deployment	A · orden C8/D10: SQL ritmo → secrets + Deploy Edge → variable Pages → web2 candidata → verificada → integrate → Deploy Web → smoke → SQL cierre REST → smoke. B · SQL columnas → Deploy Edge (confirm + drain, verificados en vivo) → cron → web2 (página de confirmación) → integrate → Deploy Web → estreno real.
Verification	DoD de A y de B arriba (técnica + Lucía y Pilar desde sus móviles + digest del día siguiente).
hub + Notion	Hub: este doc 54 + adjunto (po256); as-built al cerrar cada entrega. Notion: sin bloques → hub.

Firmas

Persona	Rol	Conforme	Fecha
Juanjo Cebrián	Ingeniería y producto (D1–D10 y D14, carril técnico)		
Pilar González	CEO · cumplimiento (FIRMA D11–D13 =  WL-2; corrobora D3: Turnstile como excepción declarada a po74; privacidad)		
Lucía Cebrián	COO · diseño (FIRMA D11–D13 =  WL-2; escribe el correo de confirmación en 5 idiomas; widget bajo el botón; mensaje «verificación pendiente»)		